# Практическое занятие № 6

# Защита данных на уровне таблиц, строк и представлений

## 1. Цель практической работы

Научиться защищать информацию в базе данных с помощью разграничения доступа на разных уровнях.

В ходе работы должны научиться:

- предоставлять и отзывать права на таблицы;
- ограничивать доступ к отдельным столбцам;
- создавать представления `VIEW`;
- разграничивать доступ к строкам с помощью RLS;
- проверять разрешённые и запрещённые операции;
- применять принцип минимальных привилегий.

## 2. Исходная ситуация

В университете используется база данных, содержащая информацию о студентах и их оценках.

В таблице `students` хранятся персональные данные, а в таблице `grades` — результаты обучения.

Необходимо настроить защиту таким образом, чтобы:

- преподаватели видели только необходимые сведения о студентах;
- финансовые данные не были доступны преподавателям;
- каждый преподаватель видел только оценки, за которые он отвечает;
- пользователи не могли изменять или удалять данные без соответствующих разрешений.

## Задание 1. Подготовка учебной базы данных

Создайте две таблицы.

### 1.1. Таблица студентов

```
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    group_name VARCHAR(50) NOT NULL,
    email VARCHAR(100),
    scholarship NUMERIC(10,2)
);
```

### 1.2. Таблица оценок

```
CREATE TABLE grades (
    id SERIAL PRIMARY KEY,
    student_id INT NOT NULL REFERENCES students(id),
    teacher_username TEXT NOT NULL,
    subject VARCHAR(100) NOT NULL,
    grade INT NOT NULL CHECK (grade BETWEEN 0 AND 100)
);
```

### 1.3. Добавление тестовых данных

```
INSERT INTO students
    (full_name, group_name, email, scholarship)
VALUES
    ('Иван Петров', 'IT-21', 'ivan@example.com', 25000),
    ('Анна Садыкова', 'IT-22', 'anna@example.com', 30000),
    ('Данияр Ахметов', 'IT-21', 'daniyar@example.com', 20000),
    ('Мария Иванова', 'IT-23', 'maria@example.com', 28000);
```

Добавьте оценки:

```
INSERT INTO grades
    (student_id, teacher_username, subject, grade)
VALUES
    (1, 'teacher01', 'Базы данных', 90),
    (2, 'teacher01', 'Базы данных', 85),
    (3, 'teacher02', 'Компьютерные сети', 92),
    (4, 'teacher02', 'Компьютерные сети', 88);
```

Проверьте данные:

```
SELECT * FROM students;
SELECT * FROM grades;
```

**Вопрос:** Какие данные в таблице `students` следует считать конфиденциальными и почему?

## Задание 2. Разграничение прав на таблицы и столбцы

## Часть А. Создание ролей

Создайте роли для преподавателя и бухгалтера:

```
CREATE ROLE teacher_limited;
CREATE ROLE accountant_limited;
```

## Часть Б. Ограничение доступа к столбцам

Преподавателю необходимо видеть идентификатор студента, его Ф. И. О. и группу.

Предоставьте права:

```
GRANT SELECT (id, full_name, group_name)
ON students
TO teacher_limited;
```

Теперь выполните проверку от имени роли с соответствующими правами:

```
SELECT id, full_name, group_name
FROM students;
```

Затем попробуйте прочитать стипендию:

```
SELECT scholarship
FROM students;
```

Ожидаемый результат: если у роли нет других прав на чтение этого столбца, PostgreSQL должен запретить запрос.

## Часть В. Права бухгалтера

Бухгалтеру необходим доступ к финансовому столбцу:

```
GRANT SELECT (id, scholarship)
ON students
TO accountant_limited;
```

**Задание:** заполните таблицу результатов.

| Операция | Ожидаемый результат |
|---|---|
| Преподаватель читает Ф. И. О. | Разрешено |
| Преподаватель читает стипендию | Запрещено при отсутствии других прав |
| Бухгалтер читает стипендию | Разрешено |
| Преподаватель изменяет стипендию | Запрещено при отсутствии права UPDATE |

Обратите внимание: право на столбец не заменяет права на другие столбцы и не предоставляет автоматически все остальные привилегии таблицы.

# Задание 3. Создание защищённого представления VIEW

## Часть А. Создание представления

Создайте представление, которое показывает только общие сведения о студентах:

```
CREATE VIEW student_public_info AS
SELECT id, full_name, group_name
FROM students;
```

Предоставьте преподавателю право чтения:

```
GRANT SELECT
ON student_public_info
TO teacher_limited;
```

## Часть Б. Проверка представления

Выполните запрос:

```
SELECT *
FROM student_public_info;
```

Ожидаемый результат: выводятся три столбца:

- `id`;
- `full_name`;
- `group_name`.

Стипендия и электронная почта в представлении отсутствуют.

## Часть B. Ограничение прямого доступа

Чтобы преподаватель использовал представление, а не исходную таблицу, необходимо проверить и ограничить прямые разрешения на `students`.

Например, если преподавательской роли ранее выдавалось право чтения всей таблицы:

```
REVOKE SELECT ON students FROM teacher_limited;
```

После этого доступ к представлению должен проверяться отдельно от прямого доступа к исходной таблице.

**Важно:** отзыв прав работает только для соответствующего источника разрешения. Если доступ наследуется через другую роль или предоставлен напрямую другому пользователю, его также необходимо проверить.

### Вопросы

1. Какие данные скрывает представление?
2. Почему `VIEW` не заменяет настройку прав доступа к исходной таблице?
3. В каких случаях представление удобнее выдачи прав на всю таблицу?

# Задание 4. Настройка Row-Level Security (RLS)

## Часть A. Создание пользователей

Выполните от имени администратора БД:

```
CREATE ROLE teacher01 LOGIN PASSWORD 'Teacher01_Strong!';
CREATE ROLE teacher02 LOGIN PASSWORD 'Teacher02_Strong!';
```

Создайте общую роль:

```
CREATE ROLE teacher_rls_role;

GRANT teacher_rls_role TO teacher01;
GRANT teacher_rls_role TO teacher02;
```

Предоставьте необходимые права на таблицу оценок:

```
GRANT SELECT, UPDATE
ON grades
TO teacher_rls_role;
```

Поскольку у таблицы `grades` есть поле `id` с типом `SERIAL`, оно использует последовательность для генерации идентификаторов. Для чтения и изменения уже существующих оценок в этом упражнении отдельное право на последовательность не требуется.

## Часть Б. Включение RLS

```
ALTER TABLE grades
ENABLE ROW LEVEL SECURITY;
```

Эта команда включает построчную защиту для таблицы.

## Часть В. Создание политики чтения

Создайте политику:

```
CREATE POLICY grades_select_policy
ON grades
FOR SELECT
TO teacher_rls_role
USING (teacher_username = current_user);
```

Условие `USING` определяет, какие существующие строки доступны пользователю для чтения.

При подключении непосредственно под пользователем `teacher01` он должен видеть строки, в которых:

```
teacher_username = 'teacher01'
```

Пользователь `teacher02` должен видеть строки, в которых:

```
teacher_username = 'teacher02'
```

## Часть Г. Создание политики изменения

```
CREATE POLICY grades_update_policy
ON grades
FOR UPDATE
TO teacher_rls_role
USING (teacher_username = current_user)
WITH CHECK (teacher_username = current_user);
```

Здесь:

- `USING` ограничивает строки, которые можно изменять;
- `WITH CHECK` проверяет, соответствует ли новая версия строки условию политики.

Эта политика не отменяет необходимость обычного права `UPDATE`.

**Важное замечание:** преподаватель не должен иметь возможность присвоить себе чужую оценку, изменив `teacher_username`. В реальной системе следует ограничить обновляемые столбцы и отдельно проверить все пути доступа к таблице.

## Задание 5. Проверка защиты данных

Проверку следует выполнять под соответствующими учётными записями PostgreSQL. Обычный запрос `SET ROLE` может использоваться для учебной проверки, но необходимо учитывать, что `current_user` при этом меняется на выбранную роль.

## Проверка № 1. Преподаватель teacher01

Подключитесь как `teacher01` и выполните:

```
SELECT *
FROM grades;
```

**Ожидаемый результат:** две записи с оценками, назначенными `teacher01`.

Затем выполните:

```
UPDATE grades
SET grade = 95
WHERE id = 1;
```

Если строка № 1 принадлежит `teacher01`, изменение должно быть разрешено при наличии всех необходимых прав.

Попробуйте изменить строку № 3:

```
UPDATE grades
SET grade = 100
WHERE id = 3;
```

**Ожидаемый результат:** чужая строка не должна измениться.

## Проверка № 2. Преподаватель teacher02

Подключитесь как `teacher02` и выполните:

```
SELECT *
FROM grades;
```

Ожидаемый результат: преподаватель видит только оценки, где `teacher_username = 'teacher02'`.

## Проверка № 3. Ограничение столбцов

Выполните от имени роли `teacher_limited` запрос:

```
SELECT scholarship
```

```
FROM students;
```

Ожидаемый результат: запрос должен быть запрещён, если роль не имеет другого разрешения на этот столбец.

## Проверка № 4. Доступ через VIEW

Выполните:

```
SELECT *
FROM student_public_info;
```

Ожидаемый результат: доступны только `id`, `full_name` и `group_name`, если представление корректно настроено и у пользователя нет иных путей получения дополнительных данных.

### Таблица результатов

| Проверка | Ожидаемый результат | Фактический результат |
|---|---|---|
| Чтение разрешённых столбцов | Разрешено | Заполнить |
| Чтение стипендии преподавателем | Запрещено | Заполнить |
| Чтение через VIEW | Разрешено | Заполнить |
| Чтение оценок teacher01 | Только свои строки | Заполнить |
| Чтение оценок teacher02 | Только свои строки | Заполнить |
| Изменение чужой оценки | Не изменяется | Заполнить |

### Что должно быть в отчёте

1. Название и цель практической работы.
2. SQL-код создания таблиц и тестовых данных.
3. Команды создания ролей и назначения прав.
4. Код создания представления `student_public_info`.
5. Код включения RLS и создания политик.
6. Таблица с результатами проверок.
7. Вывод о том, какие механизмы защиты были использованы и какие угрозы они уменьшают.

**Главный вывод:** для защиты базы данных недостаточно просто создать пользователей и назначить им роли. Необходимо определить, какие таблицы, столбцы и строки доступны каждой категории пользователей, а затем проверить, что запреты действительно работают.